Lecture 2

Handling Individual Time Steps

Output vector y- hat at a particular time step is just going to be a function of the input at the time step. >> We're going to completely ignore the inputs at the previous time steps to make a prediction at a later step because these are all treated in isolation

We want to pass information on forward through time

>> we define a variable called internal state h of t, which is maintained by the neurons in this network.

$\hat{y}_t = f(x_t, h_{t-1})$ x_t: input, h_t-1: past memory

Our prediction why h of t at a particular time step should depend not only on the input at that time step but also this state that's been carried on from the prior step

Recurrent Neural Networks (RNNs)

> Apply a recurrence relation at every time step to process a sequence.

$$\underbrace{\boxed{h_t}}_{\text{cell state}} = \underbrace{\boxed{f_W}}_{\substack{\text{function} \\ \text{with weights} \\ W}} (\underbrace{\boxed{x_t}}_{\text{input}}, \underbrace{\boxed{h_{t-1}}}_{\text{old state}})$$

RNN: Taking the last hidden state and predicting some output and updating the hidden state and doing this iteratively through this iterative procedure such that at the end we can generate a prediction for the next word that comes after we've processed these inputs> update and output

Internal state is being passed on time step to time, we're generating the predictions. We have a weight matrix that defines how the hidden state gets updated and finally one that transforms the hidden state to the ouput prediction. Weight matrices are reused at each time step in the sequences.

Loss

We apply same perceptron here, but the only difference is that we can compute a loss at each slice and get a total loss by summing over all the time steps.

How do we operationalize this in the real world?

Sequences

- handle variable-length sequences

- track long-term dependencies

- maintain information about order

- share parameters across the sequence

A sequence modeling problem: Predict the next word

Given words: This morning I took my cat for a walk

Predict the next word: walk

Core consideration: vectors and matrices and ways of representing numbers; we need a way to represent these words as numbers in a numerical representation to be able to operate on them using a neural network.

Embedding: Taking an input and then transforming it into first an index that then can map to a vector of some fixed size

Backpropagation Through Time (BPTT)

We consider an individual loss and the aggregate loss through all these individual time steps. >> we backpropagate errors across these individual time steps, such that we can carry through errors from late time steps all the way back to the beginning.

If many values > 1, exploding gradients.

If many values < 1, vanishing gradients.

Long Short-Term Memory network (LSTM)

Example task: Music Generation

Given the past history of musical notes, a model learns to predict the most likely next musical note in the sequence.

Limitations of RNNs

1. Encoding bottleneck to the amount of information that the RNN state can hold

2. Slow, no parallelization

We want to be able to process our sequence efficiently

What if we could try to tackle the sequence modeling problem without having to deal with the data time step by time step?

We can maybe eliminate the need for recurrence

We can maybe do this by squashing everything together.

We destroy all information about order.

>> No recurrence, not sclable. No order, no long memory.

Attention:

Transformer is a type of neural network architecture

Intuition behind self-attention.

Problem: the notion of being able to identify which parts are important in some input?

Goal: identify and attend to most important features in input.

1. encode position information

2. extract query, key, value for search

3. compute attention weighting

4. extract features with high attention

What does the similarity computation actually mean?

Softmax: to basically squish values between zero and one.

Attention is the foundational building block of Transformers.